



BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



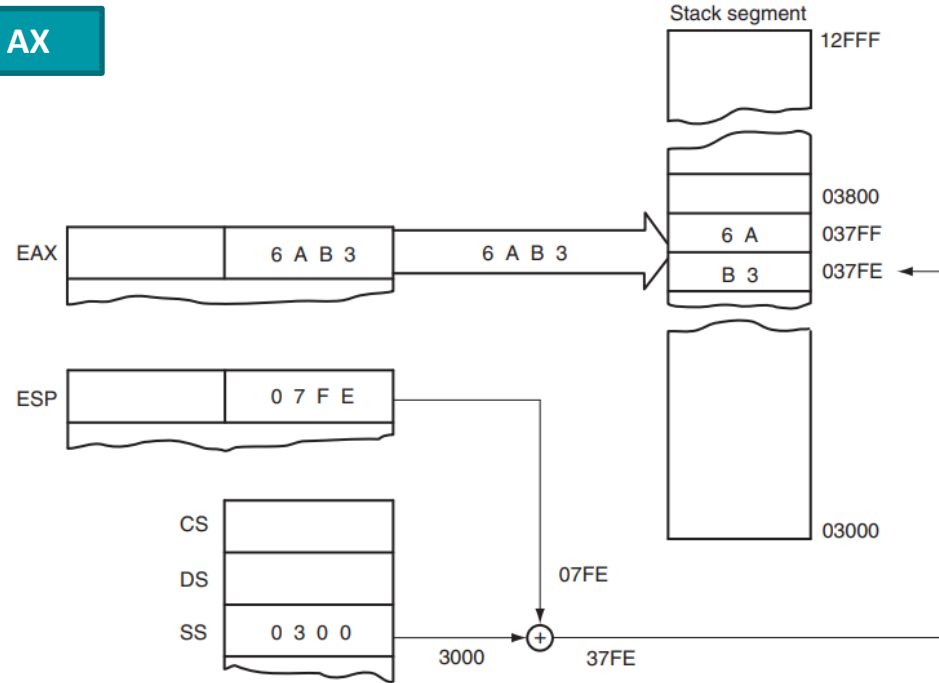
Data Movement Instructions

MOV Instructions available:

- MOV
- MOVSX -**M**ove with **S**ign-**E**xtend
- MOVZX -**M**ove with **Z**ero-**E**xtend
- When do you use MOVSX and MOVZX?
 - Need to work with values of different sizes (e.g., moving from a byte to a word) while preserving the sign (for MOVSX) or not (for MOVZX).

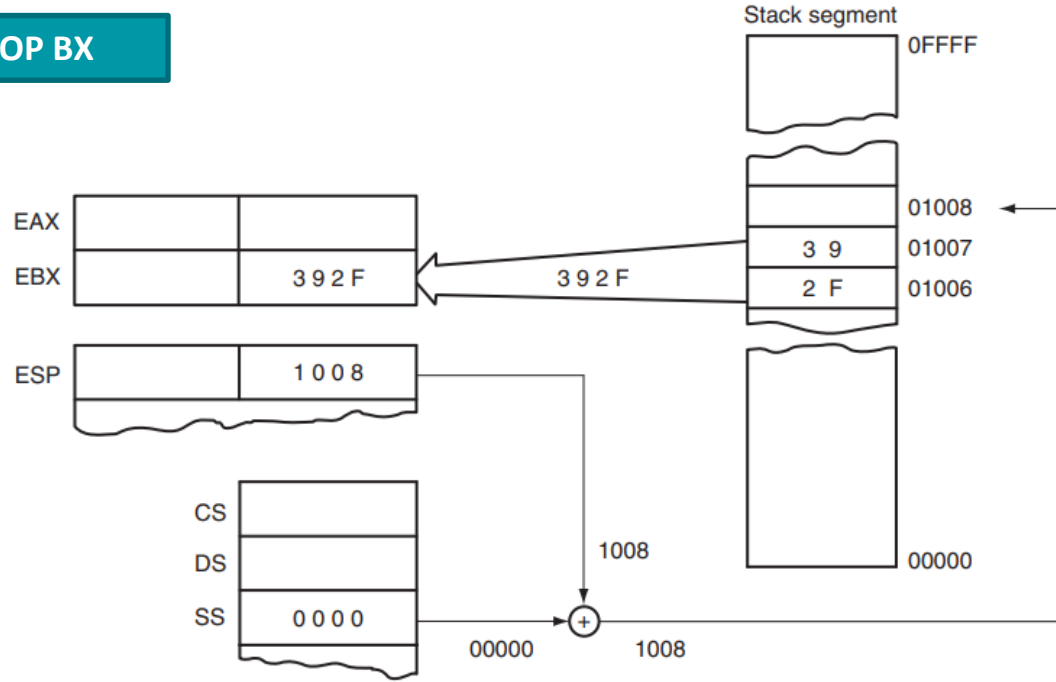
PUSH Instruction

PUSH AX



POP Instruction

POP BX



LEA- LOAD-EFFECTIVE ADDRESS

- The LEA instruction loads a 16- or 32-bit register with the offset address of the data specified by the operand.

E.g. LEA AX,NUMB

- Loads AX with the offset address of NUMB
- MOV BX,OFFSET LIST is same as LEA BX,LIST

Why is the LEA instruction available if the OFFSET directive accomplishes the same task?

- OFFSET only functions with simple operands such as LIST. It may not be used for an operand such as [DI], LIST [SI], and so on.
- It takes the microprocessor longer to execute the LEA BX,LIST instruction than the MOV BX,OFFSET LIST.
- It is because the assembler calculates the offset address of LIST, whereas the microprocessor calculates the address for the LEA instruction.
- The MOV BX,OFFSET LIST instruction is actually assembled as a move immediate instruction and hence is more efficient.

Example

E.g. LEA BX, [DI]
DI=1000H -> BX=1000H

Can this be done using MOV BX, DI ?

E.g. LEA SI,[BX+DI]
BX=3000H, DI=2000H -> SI=3000H (Sum= modulo-64)

If BX=1000H DI = FF00H , what is SI?

LEA BX, [DI] is same as MOV BX, DI
Both transfer a copy of offset
address(DI) in BX

DI – Data stored at DI
[DI] – Data stored at the address of DI

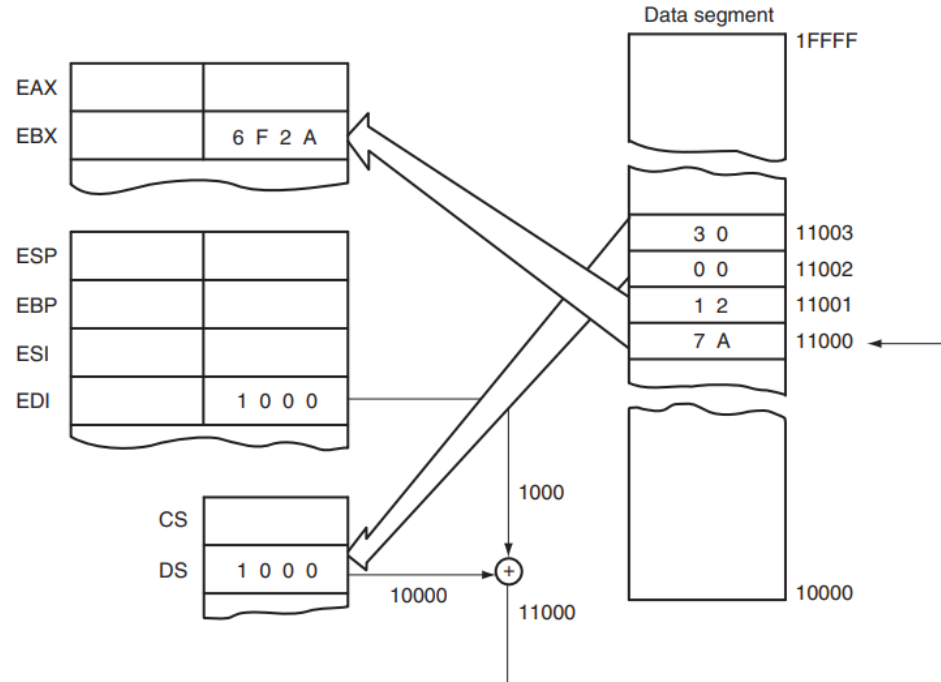
Example

```
DATA1    .DATA                ;start data segment
          DW      2000H        ;define DATA1
DATA2    DW      3000H        ;define DATA2
          .CODE                ;start code segment
          .STARTUP             ;start program
          LEA SI,DATA1         ;address DATA1 with SI
          MOV DI,OFFSET DATA2 ;address DATA2 with DI
          MOV BX,[SI]          ;exchange DATA1 with DATA2
          MOV CX,[DI]
          MOV [SI],CX
          MOV [DI],BX
          .EXIT
          END
```

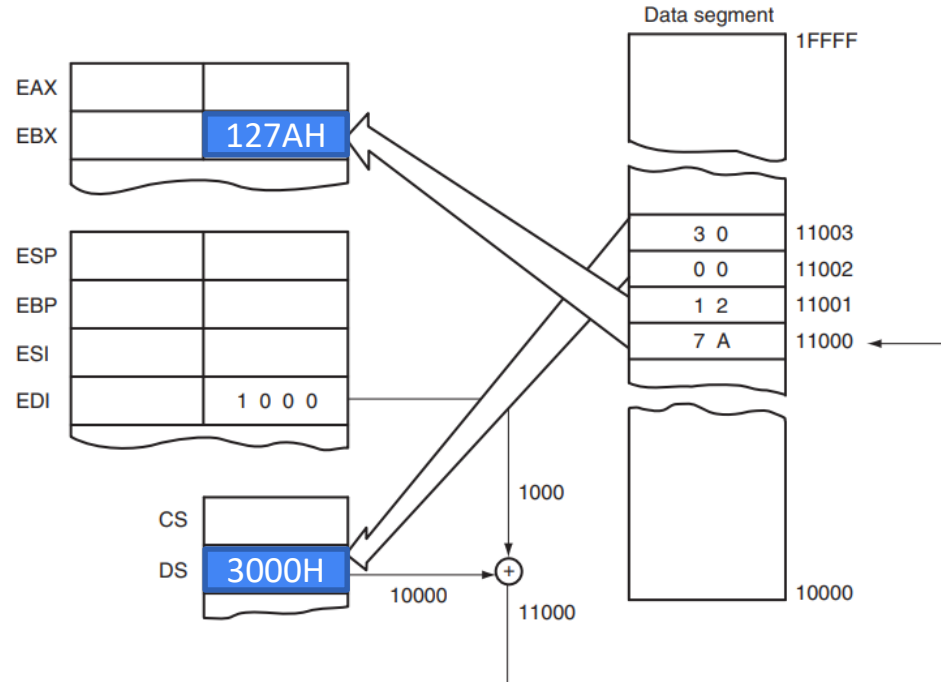

LDS, LES

- LDS (Load DS), LES (Load ES)
- Load any 16-bit or 32-bit register with an offset address, and the DS, ES, or SS segment register with a segment address.
- Used when working with data structures that include a far pointer, which consists of both a segment and an offset.
- E.g. LDS BX,[DI] – transfer 32 bit number addressed by DI in data segment into BX and DS registers.

LDS Example : LDS BX,[DI]



LDS Example : LDS BX,[DI]



Far Pointer

- A far pointer combines both the segment and offset into a 32-bit value. It is used to address memory locations beyond the 64 KB limit imposed by a single segment in the x86 architecture.
- The format of a far pointer is typically:

Far Pointer=(Segment $\ll$ 4)+Offset

STRING DATA TRANSFERS

- Five string data transfer instructions: LODS, STOS, MOVS, INS, and OUTS.
- Each string instruction allows data transfers that are either a single byte, word, or doubleword (or if repeated, a block of bytes, words, or doublewords).
- String instructions use
 - D flag-bit (direction)
 - DI and SI registers

Direction Flag

- **The Direction Flag:** (D, located in the flag register) selects the auto-increment or the auto-decrement operation for the DI and SI registers during string operations.
- CLD instruction clears the D flag and the STD instruction sets it .
- CLD instruction selects the auto-increment mode and STD selects the auto-decrement mode .

DI and SI

- Memory accesses occur through either or both of the DI and SI registers.
- The DI offset address accesses data in the extra segment for all string instructions that use it.
- The SI offset address accesses data, by default, in the data segment.
- Transferring a byte, the contents of DI and/or SI are **incremented or decremented by 1.**
- Transferring a word, the contents of DI and/or SI are **incremented or decremented by 2.**
- Transferring a Doubleword cause DI and/or SI to **increment or decrement by 4.**

LODS

- The LODS instruction loads AL, AX, or EAX with data stored at the data segment offset address indexed by the SI register.

E.g.

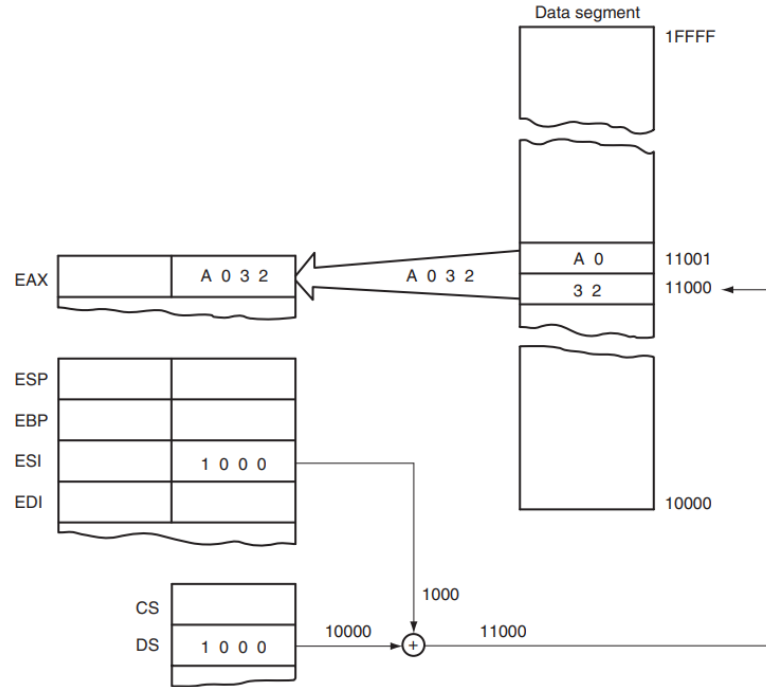
LODSB
LODSW

AL = DS:[SI]; SI = SI ± 1
AX = DS:[SI]; SI = SI ± 2

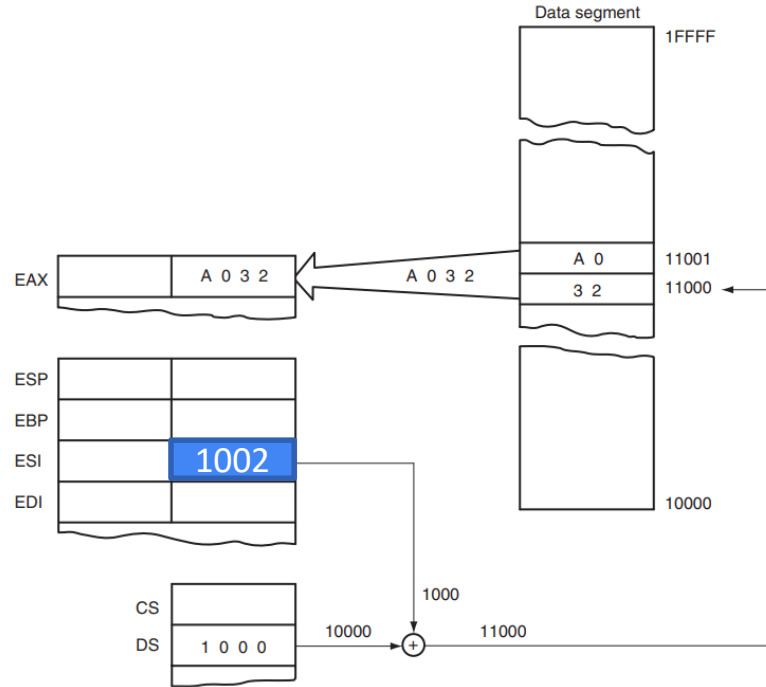
After LODSB , content of SI increment if D=0

Content of SI decrement if D=1

Example: LODSW



Example: LODSW



STOS

- Stores AL, AX, or EAX at the extra segment memory location addressed by the DI register.

E.g. **STOSB** $ES:[DI] = AL; DI = DI \pm 1$
 STOSW $ES:[DI] = AX; DI = DI \pm 2$

- The STOSB (stores a byte) instruction stores the byte in AL at the extra segment memory location addressed by DI.
- The STOSW (stores a word) instruction stores AX in the extra segment memory location addressed by DI.

Example

```
MOV AL, 10
MOV DI, 2010
STOSB
```

```

-d 2010
0859:2010  10 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0859:2020  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0859:2030  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0859:2040  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0859:2050  00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....

```

Example

- Suppose that the STOSW instruction is used to clear an area of memory called Buffer using a count called Count and the program is to function call Clear Buffer in the environment using the inline assembler.

```
void ClearBuffer (int count, short* buffer)
{
    _asm{
        push edi                ;save registers
        push es
        push ds
        mov ax, 0
        mov ecx, count
        mov edi, buffer
        pop es                  ;load ES with DS
        rep stosw               ;clear Buffer
        pop es                  ;restore registers
        pop edi
    }
}
```

The repeat prefix (REP) is added to any string data transfer instruction, except the LODS instruction. The REP prefix causes CX to decrement by 1 each time the string instruction executes. After CX decrements, the string instruction repeats. If CX reaches a value of 0, the instruction terminates and the program continues with the next sequential instruction.

MOVS

- Transfers data from one memory location to another.
- This is the only memory-to-memory transfer allowed in the 8086–Pentium 4 microprocessors.
- The MOVS instruction transfers a byte, word, or doubleword from the data segment location addressed by SI to the extra segment location addressed by DI.
- As with the other string instructions, the pointers then are incremented or decremented, as dictated by the direction flag.

	MOVS	
	MOVS	
E.g.	MOVS	
	MOVS	
	MOVS	

	ES:[DI] = DS:[SI]; DI = DI ± 1; SI = SI ± 1 (byte transferred)
	ES:[DI] = DS:[SI]; DI = DI ± 2; SI = SI ± 2 (word transferred)
	ES:[DI] = DS:[SI]; DI = DI ± 4; SI = SI ± 4 (doubleword transferred)

ADD Instruction

Adding two operands and storing the result in the destination operand.

- Syntax: ADD destination, source

- E.g.

ADD AL,BL

$AL = AL + BL$

ADD CX,DI

$CX = CX + DI$

ADD EBP,EAX

$EBP = EBP + EAX$

ADD CL,44H

$CL = CL + 44H$

ADD BX,245FH

$BX = BX + 245FH$

ADD EDX,12345H

$EDX = EDX + 12345H$

ADD Instruction

ADD [BX],AL	AL adds to the byte contents of the data segment memory location addressed by BX with the sum stored in the same memory location
ADD CL,[BP]	The byte contents of the stack segment memory location addressed by BP add to CL with the sum stored in CL
ADD AL,[EBX]	The byte contents of the data segment memory location addressed by EBX add to AL with the sum stored in AL
ADD BX,[SI+2]	The word contents of the data segment memory location addressed by SI + 2 add to BX with the sum stored in BX
ADD CL,TEMP	The byte contents of data segment memory location TEMP add to CL with the sum stored in CL
ADD BX,TEMP[DI]	The word contents of the data segment memory location addressed by TEMP + DI add to BX with the sum stored in BX
ADD [BX+D],DL	DL adds to the byte contents of the data segment memory location addressed by BX + DI with the sum stored in the same memory location
ADD BYTE PTR [DI],3	A 3 adds to the byte contents of the data segment memory location addressed by DI with the sum stored in the same location

Thank You